

E03 · 망각과 잠 — 왜 오프라인이 필요한가

이 권은 본문 ch07(망각과 replay) 과 ch08(CLS와 생물학적 sleep) 을 수식과증명 없이 다시 설명한다. 답할 질문은 둘이다. **가중치를 계속고치면 왜 예전 것이 무너지는가**, 그리고 **그 문제의 해법에 왜 “잠”이라는 이름이 붙었는가**.

두 번째 질문에는 불편한 답이 달려 있다. 그 이름을 준 생물학 논문들이 실제로 보인 것은 흔히 믿어지는 것보다 훨씬 좁고, 은유가 깨지는 지점이 분명히 있다. 이 권은 절반을 그 경계선을 긋는 데 쓴다.

1. 독자의 세계에는 망각이 없다

서빙 중인 transformer 를 떠올린다. 가중치는 움직이지 않는다. KV cache 는 append-only 여서 어제 넣은 항목이 오늘의 write 때문에 변질되지 않는다. 그래서 inference 엔지니어에게 “모델이 잊는다” 는 말은 낯설다. 잊을 자리가 없다.

이 권이 다루는 라인은 그 불변식을 깬다. 상태를 세 층으로 갈라 보면 어디가 깨지는지 정확히 보인다.

기호 풀이 Θ (slow weights) = 배포되는 가중치 그 자체. 우리가 .safetensors 로 굽는 것. W (fast weights) = 질의를 처리하는 동안 움직이는 고정 크기 상태. E (external store) = 모델 밖의 텍스트·벡터·그래프 저장소. k = **sleep 라운드** 침자 (오프라인 정리 작업의 회차). t = 토큰 침자. 이 둘은 다른 시계다.

E 를 고치는 것은 DB 에 행을 하나 더 넣는 일이고, 덮어쓰면 그만이다. W 는 질의가 끝나면 사라진다. **오직 Θ 만이 라운드마다 다른 함수가 되어 남는다.** 그리고 바로 그 층에서만 망각이 일어난다.

비유 Θ 를 고치는 것은 캐시를 비우는 일이 아니라 **모델을 재배포하는 일**이다. sleep 라운드가 하루 한 번 돈다면 1년에 365개의 배포아티팩트가 생긴다. — **비유가 깨지는 곳**: 보통의 재배포는 사람이 만든 새 체크포인트를 올리는 것이고 롤백 대상이 명확하지만, 여기서는 모델이 스스로 만든 데이터로 자기를 고쳐 놓았으므로 “무엇으로 되돌릴 것인가”의 기준선 자체가 라운드마다 흐려진다.

이 관점을 잡으면 뒤에 나오는 학술 지표 하나가 낯설지 않아진다. 이 권의 주인공인 **망각률 ρ** 는 결국 **릴리스마다 회귀스위트에서 잡히는 성능 회귀의 크기**다.

자세한 것은 ch07 § 07.1.

2. 망각은 버그가 아니라 갱신식이 정의하는 동작

catastrophic forgetting 은 과제 A 를 이미 학습한 network 를 과제 B 의 데이터만으로 계속 훈련할 때 A 의 성능이 점진적이 아니라 급격하게 — 종종 거의 전부 — 무너지는 현상이다.

왜 그런지는 Θ 를 고치는 규칙 하나만 보면 끝난다.

$$\Theta_{k+1} = \Theta_k - \eta_{\Theta} \nabla_{\Theta} \mathcal{L}(\mathcal{R}_k; \Theta_k) \quad (\text{U-}\Theta)$$

기호 풀이 $\mathcal{L}$ = loss. 모델의 출력이 목표와 얼마나 어긋나는지를 하나의수로 잰 값이며, 작을수록 좋다. $\nabla_{\Theta} \mathcal{L}$ = gradient. Θ 의 각 좌표를 아주 조금 키웠을 때 $\mathcal{L}$ 이 얼마나 커지는지를 좌표마다 적어 놓은 벡터다. η_{Θ} = sleep 라운드의 learning rate(한 걸음의 보폭). $\mathcal{R}_k$ = 라운드 k 가 실제로 학습에 쓰는 데이터 집합.

이 식이 말하는 것. loss를 가장 빠르게 줄이는 방향의 반대로그중치를 한 걸음 옮긴다. 그 걸음이 보는 것은 오직 $\mathcal{R}_k$ 안에 든 것뿐이다.

여기서 결론이 나온다. $\mathcal{R}_k$ 에 과거 과제가 하나도 들어 있지않으면, $\mathcal{L}$ 에도 과거 과제 항이 없고, 갱신은 과거를 지키려는압력을 **한 번도 받지 않는다**. 망각은 부작용이 아니라 그 경우의갱신식이 정의하는 바로 그 동작이다.

비유 하나의 점토판에 계속 새기는 것과 같다. 새 글자를 파려면 흙을 밀어야하고, 밀린 흙이 옛 글자를 메운다. 판은 늘 같은 크기다. — **비유가 깨지는 곳:** 점토판은 물리적으로 인접한 자리만 뚫개지지만 network의 지식은 수많은 좌표에 분산되어 있어 “엠티리”라는것이 없다. 한 번의 갱신이 판 전체를 조금씩 밀고, 그래서 어느 글자가지워질지 미리 짚을 수 없다.

대응책은 전부 이 한 줄 식의 어딘가를 고치는 일이다. 고칠 수 있는 자리는셋뿐이다 — $\mathcal{R}_k$ (무엇을 보여 줄 것인가), $\mathcal{L}$ (무엇을 벌할 것인가), **갱신 방향** (어디로갈 것인가).

자세한 것은 ch07 § 07.1.

3. 대응책 네 가족 — 한표로

표 E03-1. 망각 대응 네 가족과 각각이 고치는 자리.

가족	고치는 자리	한 줄 요약	라운드가 쌓이면
rehearsal	$\mathcal{R}_k$	과거 실제 예제를 보관했다가 섞는다	저장량이 과제 수에 비례해 자란다
generative replay	$\mathcal{R}_k$	과거 데이터를 생성기 로 바꿔 다시 만들어 낸다	저장은 고정, 생성 계산이 매 라운드 든다
EWC	$\mathcal{L}$	중요한 좌표를 옛 값에 스프링으로 묶는다	비용이라운드 수와 무관
GEM	갱신 방향	과거를 해치는 방향이면 방향을 꺾는다	step 비용이라운드 수에 선형

네 방법의 공통점 하나가 systems 관점에서 가장 중요하다. **넷 다서빙 비용이 똑같다**. 최종 산출물은 어느 쪽이든 평범한 가중치 벡터하나이고, 읽기는 $\hat{y} = f(q; \Theta)$ 한 번의 forward다. per-user delta가 생기지 않으므로 shared-weight batching도 그대로 성립한다. 차이는전부 **오프라인 라운드 안에서** 어느 자원을 태우느냐다.

기호 풀이 L_w = wake 지연 기여 (ms, TTFT/ITL). B_s = sleep 라운드 하나의 계산 예산 (FLOPs 또는 tokens). C_{cap} = 상태용량 (bytes). ρ = 망각률 (라운드당 성능 손실). 이 라인의 논문은 전부 이 네 칸으로 환원해 읽는다.

자세한 것은 ch07 § 07.2 와 ch07 (state, update, cost) 정리.

4. EWC — 중요한 좌표를 스프링으로 묶는다

EWC(elastic weight consolidation; Kirkpatrick et al., arXiv:1612.00796) 는 과거 데이터를 **하나도 남기지 않는** 쪽을 택한다. 대신 라운드가 끝난 순간의 가중치를 사진처럼 찍어 두고, 다음 라운드에서 그 사진으로부터 멀어지는 것에 벌점을 매긴다.

$$\mathcal{L}_{\text{EWC}}(\Theta) = \underbrace{\mathcal{L}(\mathcal{R}_k; \Theta)}_{\text{새 과제}} + \underbrace{\sum_i \frac{\lambda}{2} F_i (\Theta_i - \Theta_i^*)^2}_{\text{과거를 지키는 벌점}}$$

기호 풀이 Θ^* = **anchor**. 직전 라운드가 끝난 순간의 가중치 스냅샷. Θ_i = 가중치의 i 번째 좌표 (스칼라 하나). F_i = 그 좌표의 **중요도**. λ = 옛 과제와 새 과제의 저울 (클수록 보수적). $\sum_i$ = 모든 좌표에 대한 합.

이 식이 말하는 것. 새 과제를 잘하되, **중요한 좌표**일수록 옛 값에서 멀어지는 것을 비싸게 만든다. 중요하지 않은 좌표는 마음대로 움직여도 된다.

중요도 F 를 어떻게 아는가. EWC 는 **Fisher 정보**의 대각을 쓴다 — 그 좌표를 흔들었을 때 과거 과제의 예측이 얼마나 흔들리는지를 재는 국소 통계다. 유도는 Bayes 규칙과 Laplace 근사 두 단계로 나오며, 그 계산은 본문에 있다.

비유 논문 자신의 비유다. 각 가중치를 옛 값에 **스프링**으로 묶되, 스프링 상수를 좌표마다 다르게 준다. 중요한 좌표는 뻣뻣한 스프링, 나머지는 헐렁한 스프링. — **비유가 깨지는 곳**: 진짜 스프링은 늘어난 만큼 되돌리지만 이 벌점은 **직전 anchor** 하나만 기억한다. 과제가 셋 이상 쌓이면 스프링을 과제마다 따로 달 것인지 하나로 합칠 것인지가 갈리고, EWC 논문은 실험에서 어느 쪽을 썼는지 밝히지 않았다 [EWC § 2].

시스템 모델링 관점 벌점 항을 미분하면 갱신에 더해지는 것은 $\lambda F \odot (\Theta - \Theta^*)$ 하나뿐이다 — 전체 파라미터 벡터에 대한 elementwise 곱셈-덧셈 한 번, 즉 **AXPY** 다. FLOP-bound 가 아니라 **대역폭-bound**이며, step마다 F 와 Θ^* 두 벡터를 더 읽으므로 $2|\Theta|$ 개 원소의 추가 트래픽이 붙는다. 훈련 측 상주 상태는 논문 표현으로 시냅스당 세 값 ($3|\Theta|$) 이고 dtype·바이트는 본문에 없다. 결정적인 것은 이 비용이 **라운드 수 k 와 무관한 상수**라는 점이다. 매일자는 시스템에서 이 성질은 사치가 아니라 요구사항이다. 그리고 서버 쪽 추가 비용은 정확히 **0** 이다 — 벌점은 훈련 목적 함수에만 있고 평가는 훈련을 끈 채로 한다.

여기서 이 권이 반드시 넘겨야 할 사실이 있다. EWC 논문에는 결과표가 없다. 논문의 표 두 개는 둘 다 하이퍼파라미터 표이고, EWC의 정확도 수치는 본문 어디에도 인쇄되어 있지 않다. 전 파일에서 유일한정량적 성능 진술은 baseline 이 “총 human-normalized 점수 1 미만” 이라는것이다 [EWC § 2.2]. 헤드라인은 “많은 수의 과제를 순차 학습하며 오차율은 완만하게만 증가한다” 인데 **과제가 몇 개인지가 본문에 없다** — 그 수는 라벨 없는 x 축 위에만 있다.

한계 논문은 자기 근사의 절반을 스스로 반증하고 그것을 적는다. Fisher 가 “중요하지 않다” 고 판정한 방향으로 가중치에 잡음을 넣으면 성능이 변하지 않아야 하는데, 실제로는 중요한 방향의 잡음과 **같은 정도로 떨어진다**. 논문의 문장은 “우리가 특정 파라미터가 중요하지 않다고 과신하고 있음을 시사한다” 이다 [EWC § 2.2]. 이후 문헌에 인쇄된 “EWC = X%” 는 전부 **재구현에서 나온 값**이며, 이 권의 § 5 에 나오는 EWC 수치도 GEM 저자들의 재구현이다.

자세한 것은 ch07 § 07.3 과 § 07.3.1.

5. GEM — 과거를 해치는 방향이면 꺾는다

GEM(Gradient Episodic Memory; Lopez-Paz & Ranzato, arXiv:1706.08840, NIPS 2017) 은 세 번째 자리를 건드린다. $\mathcal{R}_k$ 도 $\mathcal{L}$ 도 그대로 두고 **갱신방향**을 고친다.

과거 과제의 실제 예제 일부를 과제별 buffer 에 보관한다. 그러나 **그것으로 학습하지 않는다**. 매 step 에서 두 가지를 계산한다 — 지금 배우려는 방향 g 와, 과거 buffer 가 원하는 방향 g_j . 두 방향의 사이각이 90 도 이내면 (내적이 0 이상이면) 그대로 간다. 90 도를 넘으면 조건을 만족하는 가장 가까운 방향으로 **사영**해서 간다.

기호 풀이 g = 현재 라운드 데이터에서 계산한 gradient. g_j = 과거 과제 j 의 buffer 에서 계산한 gradient. $\langle g, g_j \rangle$ = 두 벡터의 내적, 즉 방향이 얼마나 같은 편인지. M = buffer 전체 예산 (실험값 5120 개 예제).

이 식이 말하는 것. 조건 $\langle g, g_j \rangle \geq 0$ 은 “이걸음이 과거 과제의 loss 를 올리지는 않는다” 의 국소 근사다. 등식이 아니라 부등식이라는 점이 요점이다 — 과거 성능이 좋아지는 것은 막지 않는다. 논문은 이것을 자기 이점으로 명시한다: 양의 backward transfer 가 허용된다.

수치는 이 권에도 필요하다. 20 개 과제 permuted MNIST 에서 최종 정확도와 backward transfer 는 baseline 0.6018 / -0.1980, EWC 0.6185 / -0.1653, GEM 0.8260 / +0.0247 이다 [GEM App. B.1]. CIFAR-100 에서는 baseline 0.4631 / -0.1226, EWC 0.4984 / -0.0799, GEM 0.6783 / +0.0042 다 [GEM App. B.3]. GEM 만이 음수 아닌 backward transfer 를 낸다.

시스템 모델링 관점 여기서 두 개의 시계를 갈라야 한다. **step** s = 가중치를 한걸음 옮기는 단위, **라운드** k = 오프라인 정리 작업 한 회차. 한 라운드는 수천 번의 step 으로 이루어진다. 비용 구조가 EWC 와 근본적으로 다른 지점이 이 자리다. GEM 은 **step** 마다 과거 과제 buffer 전부에 대해 gradient 를 한 번씩 더 계산한다. 라운드가 k 개 쌓였

으면 step 하나에 backward pass (gradient 를 실제로 계산하는 역방향 계산) 가 k 번 붙는다 — step 비용이 **라운드 수에 선형**이다. 논문 자신이 이것을 병목으로 지목하고, CPU 훈련 시간이 그것을 보인다: permuted MNIST 에서 baseline 11 초, EWC 179 초, GEM 77 초 [GEM Table 1]. 매일 도는 sleep 라운드로 운영하면 k 는 무한정 자라고 라운드당 예산 B_s 도 함께자란다. 이 축에서는 EWC 의 상수 비용이 정확히 옳은 설계다.

한계 GEM 이 저장하는 것은 EWC 가 “과제 수에 비례해 자라서 비현실적”이라며 거부한 바로 그 **실제 예제**다. GEM 은 EWC 의 용량 논거를 논박한 것이 아니라 **다른 자원**으로 **옳긴 것**이다 — 저장은 다시 늘고, 비용은 라운드 수에 실린다. 게다가 GEM 논문이 인쇄한 backward transfer 값은 자기가 정의한 식의 분모 ($K - 1$, K 는 과제 수) 가 아니라 K 로 나눈 값이다. 순위와 부호는 하나도 바뀌지 않지만, 인쇄값에서 라운드당 값을 역산하려면 이 분모를 먼저 맞춰야 한다.

GEM 이 이 권에 반드시 있어야 하는 진짜 이유는 알고리즘이 아니다. **계기**, 즉 망각을 재는 도구다. 다음 절이 그것을 받는다.

자세한 것은 ch07 § 07.4.

6. generative replay — 데이터를 그림 그리는 사람으로 바꾼다

DGR (Deep Generative Replay; Shin et al., arXiv:1705.08690, NIPS 2017) 은 저장소 문제에 정반대로 답한다. replay 를 유지하되 **저장되는 대상을 데이터셋에서 생성기로 바꾼다**.

한 라운드가 끝나면 두 개를 들고 다음 라운드로 넘어간다 — 과거 입력 분포를 흉내 내는 **생성기**, 그리고 직전 라운드의 **모델 자신**. 다음 라운드의 학습 집합은 이렇게 만든다. 생성기가 가짜 입력을 그린다. 그 입력을 직전 모델에 넣어 나온 출력을 정답 자리에 붙인다. 그쌍을 새 실제 데이터와 비율 ω 로 섞어 학습한다.

기호 풀이 ω = 한 배치에서 새 실제 데이터가 차지하는 비율. 나머지 $1 - \omega$ 가 생성된 replay 다. 논문은 이 쌍 (생성기 + 모델) 을 **scholar** 라 부른다 — 배우면서 동시에 가르치므로 teacher 도 student 도 아니라는 뜻이다.

정답 label 이 아니라 **직전 모델의 출력**이 목표라는 점이 핵심이다. 한 모델의 출력을 정답 삼아 다른 모델을 가르치는 것을 **distillation** 이라 하고, 교사가 자기 자신의 직전 판본이면 **self-distillation** 이다. 즉 generative replay 는 언제나 self-distillation 의 한 형태다.

비유 사진첩을 버리고 **화가**를 고용하는 것이다. 앨범 (원본 데이터) 은 자리를 많이 먹고 남에게 보여 주기도 곤란하지만, “그때 그 장면” 을 그려낼 줄 아는 화가는 크기가 고정이다. — **비유가 깨지는 곳**: 화가는 자기가 본 진짜 장면을 기억하지만 이 생성기는 **자기 전임 생성기가 그린 그림**을 보고 배운다. 재귀가 이중이다 — solver 도 생성기도 자기 전임 자에게서 배우고, 외부 앵커는 매 라운드의 새 실제 데이터 ω 몫뿐이다. 복사본의 복사본을 반복하는 구조이므로 이 라인은 곧바로 model collapse 질문과 만난다 (→ ch06).

DGR 이 그 표류에 대해 내놓은 증거는 딱 한 표다. 실제 데이터로 배운 첫모델부터 이전 scholar 에게서만 배운 다섯 번째 모델까지 MNIST 정확도가 98.81% → 98.64% → 98.58% → 98.53% → 98.56%다 [DGR Table 1]. 네 세대누적 -0.25pp 이고 마지막 단계는 비단조다.

한계 이 표는 **지속학습 결과가 아니다**. 과제가 하나뿐이고 다섯 모델전부를 같은 전체 테스트 집합에서 쟀다. 측정된 것은 “한 과제에 대해자기복제 사슬이 무슨실인가” 이지 “과제들이 이어질 때 무엇을 잇는가” 가아니다. 논문의 나머지 주요 실험은 **전부 곡선이 고 숫자가 하나도없다**. 그리고 generative replay 는 자기 상한 대조군 (실제 과거 입력 + 직전모델의 예측) 을 이긴 적이 없고 비길 뿐이다. 저자들은 대가도 직접적는다 — replay 없이 훈련한 모델이 새 과제에서는 더 낮다. EWC 는 논문 전체에서 길게 논의되지만 baseline 으로 한 번도 돌지않았다. 생성기가 분포를 얼마나 복원했는지를 재는 지표 (FID 등) 는어디에도 없고, 생성기 품질은 표본 격자를 눈으로 보는 것으로만 평가된다.

시스템 모델링 관점 DGR 의 오프라인 작업은 셋으로 쪼개진다 — (i) 생성기에서의 대규모 표본생성, (ii) 동결된 직전 모델의 forward pass 로 목표 만들기, (iii) 섞인집합 위에서 모델과 생성기를 각각 재훈련. 셋 다 지연 SLA 가 없는 throughput-bound 배치 작업이고, 서버 fleet 의 비수기 용량이 자연서식지다. 저장 쪽에서는 과거 데이터 plane 이 통째로 사라진다 — 저장도 I/O 도 0 이 되고 대신 GPU 시간이 든다. 이것이 이 라인의 첫번째 명시적 교환인 **compute-for-storage** 이며, 논문은 양쪽 어느값도 매기지 않았다.

자세한 것은 ch07 § 07.5.

7. ρ — 망각을 재는 자를 먼저만든다

지금까지 세 방법이 전부 “망각을 줄인다” 고 주장했다. 그 주장을 검증할 수있었던 것은 GEM 하나다. GEM 만이 계기를 정의했기 때문이다.

계기는 표 하나에서 나온다. 성능 행렬 Π 를 만든다.

기호 풀이 $\Pi_{k,i}$ = 라운드 k 가 끝난 직후, 과제 i 에서 쟀 1 차지표 (정확도 · EM · pass@1 중 하나로 고정). 행이라운드, 열이 과제다. 대각 $\Pi_{i,i}$ 는 그 과제를 **딱 마쳤을때**의 값이며 이것이 기준점 (anchor) 이다.

ρ 는 세 질문에 답해야 정의된다.

1. **무엇을 재는가** — 이미 학습한 과제만이다. 아직 안 본 과제의향상은 망각이 아니라 forward transfer 다. 기준점은 무작위 초기화 값이아니라 $\Pi_{i,i}$ 다. 무작위 초기화를 기준으로 잡으면 망각이 아니라순 학습량을 재게 된다.
2. **언제 재는가** — 라운드 k 의 갱신이 끝난 직후, 다음 라운드가시작하기 전. 첨자는 k (sleep 라운드) 이지 t (토큰) 가 아니다.
3. **무엇에 대해 정규화하는가** — 라운드 수로 한 번, 각 과제 자신의기준값으로 한 번. 본문의 기본형은 둘 다 적용한**증분 · 상대형**이다.

핵심 망각률 ρ_k 는 sleep 라운드 하나가, 그때까지 이미 학습한 과제들에, 각 과제 자신의 기준값 대비 입힌 상대 성능 손실의 평균이다. 양수면 열화, 음수면 양의 backward transfer 다.

손으로 해보기

GEM 논문 부록의 실제 행렬에서 왼쪽 위 3×3 블록만 가져온다 (20 개 과제 permuted MNIST, 아무 대응도 하지 않는 baseline). 굵은 값이 anchor 다.

표 E03-2. GEM baseline 의 성능 행렬 일부 [GEM App. B.1.1]. 행렬 값만 논문 수치이고 아래 계산은 본문의 예시다.

	과제 1	과제 2	과제 3
라운드 1 후	0.7838	0.1264	0.0709
라운드 2 후	0.7374	0.7939	0.0981
라운드 3 후	0.6564	0.7196	0.7783

아직 학습하지 않은 과제가 0.07–0.13 인 것은 10 개 클래스의 무작위 수준이다.

라운드 2. 이미 학습한 과제는 1 번 하나다. $(0.7838 - 0.7374)/0.7838 = 0.0592$. 라운드 하나가 과제 1 을 상대 5.92% 깎았다.

라운드 3. 이제 둘이다. $(0.7374 - 0.6564)/0.7838 = 0.1033$ 과 $(0.7939 - 0.7196)/0.7939 = 0.0936$ 의 평균은 0.0985 다.

한 줄 요약 ρ 는 상수가 아니다. 5.92%에서 9.85%로 올랐다. 그러므로 ρ 를 스칼라 하나로 보고하려면 어느라운드의 값인지를 함께 적어야 한다.

같은 실행에서 “라운드당 망각” 을 부르는 방식이 네 가지고, 값이 4 배 넘게 벌어진다 — 20 라운드 총계를 순진하게 나누면 1.04pp, 과제마다 노출된 라운드 수를 보정하면 2.08pp, 라운드 2 가 실제로 입힌 낙폭은 4.64pp, 과제 1 을 끝까지 따라가면 라운드당 평균 1.66pp 다. 열화가 앞쪽 라운드에 몰려 있기 때문이다. 결론은 하나다.

핵심 정의를 밝히지 않은 ρ 는 숫자가 아니다. 값과 함께 과제집합 · 측정 시점 · 기준점 셋을 같이 보고해야 한다.

ρ 가 필요한 자리는 상각식이다. 오프라인 비용을 질의 수로 나눠 값의회계는 sleep 라운드 **한 번**의 것이고, 라운드가 K 번 반복되면 비용은 K 배가 되는 동안 품질은 ρ 만큼씩 깎인다. ρ 를 모르면 “ K 를 늘리면 이득이 쌓인다” 와 “ K 를 늘리면 모델이 무너진다” 중 어느 쪽인지 판정할 수 없다.

한계 비용 네 칸 중 ρ 가 가장 자주 비어 있다. EWC 는 ρ 를 숫자로 전혀 보고하지 않고, DGR 이 인쇄한 유일한 수열은 지속학습 설정이 아니다. **비교 가능한 ρ 가 없다는 것 자체가 본문의 발견 중 하나**이며, 그것을 발견으로 부를 수 있는 이유는 정의를 먼저 고정했기 때문이다.

자세한 것은 ch07 § 07.6 과 Worked micro-example.

8. 왜 시스템이 둘이어야 하는가 — CLS

여기서 이 권의 후반이 시작된다. 지금까지는 “어떻게 덜 잊을 것인가” 였고, 이제부터는 “왜 그 일을 **오프라인**에 해야 하는가” 다. 그 논거의원출처가 뇌과학이다.

CLS(Complementary Learning Systems)는 하나의 학습 시스템이 빠른개별 경험 기억과 느린 일반화를 동시에 감당할 수 없으므로 뇌가 두 개의상보적 시스템을 가진다고 설명하는 이론이다. McClelland, McNaughton, O’ Reilly 의 1995 년 논문이 원전이며, 이 라인의 논문들은 그것을 이렇게요약한다 — **hippocampus** 는 최근 경험을 빠르게 고해상도로 담는임시 저장소이고, **neocortex** 는 여러 경험에 걸친 규칙과 의미지식을 천천히 배우는 장기 저장소다 [LM Need Sleep § 2].

여기서 흔한 오해를 먼저 끊는다.

핵심 CLS 는 속도 논증이 아니라 간섭 논증이다. 두 시스템을 두는 이유는하나가 느려서가 아니다. 빠르게 쓰는 시스템은 새 항목이 옛 항목을덮어쓰는 것을 감수해야 하고, 느리게 쓰는 시스템은 통계를 뽑아내는 대신한 번의 경험을 즉시 담지 못한다. **두 요구를 한 파라미터 집합에있으면 둘 다 망가진다.** CLS 는 그 충돌을 아키텍처로 푼 것이다.

세 층 프레임으로 옮기면 hippocampus 는 E (external store) 자리에, neocortex 는 Θ (slow weights) 자리에 놓인다. 빠른쓰기·낮은 간섭·제한된 용량이 E 의 성질이고, 느린 쓰기·높은 일반화·배포 아티팩트의 변경이 Θ 의 성질이다. 단, **이 대응은 본문의 재구성이지 CLS 의주장이 아니다.** CLS 는 1995 년의 심리학 이론이고 external store 도 slow weights 도 그 어휘에 없다. 대응은 세우되 대응을 증거로 쓰지 않는다.

CLS 가 남긴 실용적 처방은 **interleaving** 이다 — 여러 과제를 하나의학습 집합에 섞어 제시하면 순차 학습의 덮어쓰기를 피할 수 있다. 그런데여기에 이 라인 전체를 지배하는 잔여물이 남는다.

한계 interleaving 은 작동하지만 **과거 훈련 데이터의 영구 저장을요구한다.** CLS 는 뇌가 그 저장 **없이** 같은 일을 한다고 말했고, interleaving 은 그 주장을 저장이 있는 형태로만 재현했다 [PLOS Sleep Introduction]. LLM 으로 옮기면 사용자마다 원본 대화를 통째로 보관한다는뜻이 되고, 이것은 성능 문제가 아니라저장·프라이버시·삭제 요청 대응 문제다. 이후모든 replay 계열이 하려는 일은 이 C_{cap} 을 줄이거나 없애는것이다.

자세한 것은 ch08 § 08.1.

9. 잠은 하나가 아니다 — SWS 와 REM

SWS(slow-wave sleep)는 NREM 수면의 가장 깊은 단계로, 동기화된고진폭·저주파 신경 활동으로 특징지어진다. REM(rapid eye movement) 수면은 각성에 가까운고주파·저진폭 뇌파로 특징지어지고 꿈과 가장 강하게연관된다.

배정되는 기능이 다르다. SWS 에는 두 가지가 배정된다 — 각성 중 늘어난 연결강도를 전역적으로 내리는 **synaptic homeostasis**, 그리고 최근의취약한 경험을 안정된 장기 지식으로 바꾸는 **consolidation** 이다. 그리고 그 consolidation 은 원본을 그대로 재생하는 것이 아니라 지식을**재구성**해 추상을 뽑아내는 것으로 서술된다. REM 에는 다른 기능이배정

된다 — 새로 형성된 연결의 선택적 강화와, 새 정보를 기존 의미망에 통합하는 일이다 [LM Need Sleep § 2].

이 문단이 systems 독자에게 가장 값싼 이득이다.

시스템 모델링 관점 SWS와 REM은 “sleep 라운드 하나”가 아니라 목적함수가 다른 두라운드다. 프레임 어휘로는 하나의 라운드 k 안에서 $\mathcal{L}$ 이 두 번 바뀐다. 그래서 생물학을 실제로 인용하는 논문들은 예외 없이 두 개 이상의 오프라인 목적함수를 갖는다. 반대로 sleep을 이름으로만 쓰는 논문은 오프라인 위상이 하나다. 오프라인 위상의개수는 은유가 실제로 이송되었는지를 재는 값싼 판별기다. 스케줄러 설계로 옮기면 이것은 “야간 배치 job이 하나인가 둘인가, 두 job의 목적이 서로다른가”라는 질문이 된다.

여기서 못 박아 둘 것이 하나 있다. SWS/REM 분업 서술의근거는 신경과학 실험 문헌인데, 이 라인이 실제로 손에 쥐고 있는 것은 두편의 계산 모델이다. 다음 두 절이 그 둘이며, 둘다 동물 실험이 아니라 시뮬레이션이다.

자세한 것은 ch08 § 08.2.

10. 생물학이 실제로 보인 것 — PLOS 실험

오프라인 위상이 각성 위상으로는 만들 수 없는 가중치 상태를 만든다는 것을가장 직접적으로 보인 것은 Golden·Delanois·Sanda·Bazhenov의 PLOS Computational Biology 논문 (2022) 이다.

설정은 작다. spiking 뉴런 842 개, 입력 7×7 · 은닉 28×28 · 출력 3×3 이며, 계속 학습되는 시냅스는 은닉→출력의 흥분성 6272 개뿐이다. 과제는 두 개의 상보적 foraging 과제이고, 지표는 pattern discriminability로 무작위 수준이 0.5다. 모든 수치는 10 회 이상 시행의평균 $\pm$ 표준편차다.

표 E03-3. PLOS Sleep의 핵심 네 조건 [PLOS Sleep Results].

조건	Task 1	Task 2	과거 데이터 저장
Task 1 만 학습한 직후	0.70 ± 0.02	0.53 ± 0.02	—
순차 학습 (T1 → T2)	0.52 ± 0.02	0.69 ± 0.03	불필요
고전 interleaving(T1과 T2를 섞어 학습)	0.68 ± 0.03	0.65 ± 0.04	필요
sleep 교대 (T2 학습과 sleep을 번갈아)	0.70 ± 0.03	0.68 ± 0.05	불필요

순차 학습은 옛 과제를 무작위 수준으로 떨어뜨린다 ($0.70 \rightarrow 0.52$). 새 과제 학습과 sleep을 번갈아 돌리면 옛과제가 전혀 떨어지지 않는다 ($0.70 \rightarrow 0.70$). 그리고 결정적으로, 논문의 문장 그대로 “Task 1에 대한 훈련은 어느 시점에도 수행되지 않았다.” 옛 과제데이터는 한 번도 다시 제시되지 않았다.

기제에 대한 논문 자신의 설명은 이렇다. 각성 중에는 입력층이 현재 과제의통계를 은닉층에 강제하므로 형성되는 상관관계는 새 과제의 것이다. 입력을 끊고 무구조 noise로 대체해야만 가중치 행렬 자신이 어떤상관을 되살릴지 고를 수 있고, 그때 강한 시냅스가 우선적으로 재활성화되어더 강해진다. 그 선택은 가중치만의 성질이므로 각성 절차로는 재현되지않는다.

비유 손님이 계속 들어오는 동안에는 매장을 재배치할 수 없고, 문을 닫아야진열대를 옮길 수 있다. — **비유가 깨지는 곳:** 문을 닫는 것 자체가 이득을 주는 게 아니다. 이 모델에서 이득은 “입력이 끊긴 상태에서 무엇이 스스로 재활성화되는가”에서 나오며, 그 재활성화의 내용이 옳은지를 논문은 한 번도 검사하지 않는다. 유희 시간을 확보하는 것과 유희 시간에 옳은 일을 하는 것은 다른 문제다.

이제 이 권이 반드시 넘겨야 할 반대편 사실이다.

한계 강한 baseline 이 방법과 거의 같다. 데이터를 저장하는 고전 interleaving 은 0.68/0.65 이고 sleep 은 0.70/0.68 이다. 차이는 0.02 와 0.03 인데 표준편차가 0.03–0.05 다. 논문은 본문에서 sleep 이 고전 interleaving 을 “넘어선다” 고 쓰지만, Discussion 에서 스스로 정정한다 — “고전 interleaving 이 우리 모델에서 유사한 성능을 보였지만, 우리는 후자가 **다음의 이론적 근거에서** 우월하다고 믿는다.” 이어지는 근거는 전부 이론적이며, replay 의 내용을 실제로 분석한 실험은 이 논문에 없다.

핵심 이 논문이 **실증한** 우위는 정확도가 아니라 **저장의 제거**다. 과거 데이터가 이미 있다면 이 논문은 sleep 을 택할 정확도상의 이유를 하나도 제공하지 않는다. Θ -경로를 옹호할 때 이 논문을 근거로 들 수 있는 것은 “가중치만으로 학습 집합을 만들 수 있다” 까지 이고, “그렇게 만든 학습 집합이 저장된 데이터보다 낫다”까지는 아니다. 두 문장을 붙여 쓰는 서술은 결함이다.

남은 제약도 같이 적는다. 이 논문에는 **어떤 통계 검정도 없다** — “유의하게” 라는 단어를 최소 세 번 쓰면서 검정 이름도 p-값도 없다. 과제는 **두 개뿐**이고 같은 어휘에서 뽑은 대칭 과제이며, 따라서 반복라운드에 걸친 열화율 ρ 는 **측정되지 않았다** (세 번째 과제도, 두 번째 sleep 교대 위상도 없다). 보호되는 성능 밴드는 0.5 에서 0.70 까지 폭 0.20 인데 표준편차가 그 밴드의 10–25%다. 벤치마크·환경·지표가 모두 저자제작이다. 그리고 sleep 은 **복원하지 않고 보존한다** — 논문 자신의 문장으로, 새 과제 학습 한 번이 옛 기억을 이후의 sleep 으로도 되살릴 수 없는 지점까지 지워 버릴 수 있다.

시스템 모델링 관점 이 논문은 FLOPs·wall-clock·메모리·에너지를 하나도 보고하지 않는다. 예산 B_s 는 계산량이 아니라 시뮬레이션 시간으로만 존재하고, 한 sleep 구간은 직전 각성 구간과 정확히 같아 duty ratio 가 **1:1** 이다. 그러나 값싼 구조적 사실이 하나 남는다 — 각성에서 sleep 으로 넘어가는 상태는 은닉 뉴런 784 개의 목표 발화율 스칼라 뿐이고, 통제 실험에서 그것을 **집단 평균 하나로 축약해도 결과가 유지된다** ($0.67 \pm 0.05 / 0.69 \pm 0.03$). fp32(4 B/elem) 로 가정하면 784 개는 약 3.1 KiB 이고 축약본은 4 B 다 (이 환산은 본문의 산술이며 논문 값이 아니다). 환경 전체를 보관하는 것과 3.1 KiB 를 넘기는 것은 자릿수 비교조차 되지 않는다. **여기서 교환된 것은 sleep $\leftrightarrow$ wake 가 아니라 계산 $\leftrightarrow$ 저장이다.** duty ratio 1:1 이므로 총계산은 두 배가 되고, wake 지연은 그 대로이며, 저장 바이트가 환경 전체에서 킬로바이트 이하로 바뀐다. 상각식으로 손익분기를 구할 수는 없다 — 이 논문에는 질의 스트림도 사용자도 없어 한 문맥을 공유하는 질의 수 N_q 가 정의되지 않는다.

계보상의 공백도 하나 기록한다. “저장 없이 학습 집합을 만든다” 는 목표에는 두 개의 독립된 해법이 있다 — 생성 모델을 따로 학습시키는 것 (§ 6 의 DGR) 과 모델 자신을 noise 로 돌리는 것 (이 논문) 이다. **PLOS Sleep 은 DGR 을 인용하지 않는다.** DGR 은 2017 년이고

이 논문은 2022 년이므로 연대가 강제한침묵이 아니다.
자세한 것은 ch08 § 08.3.

11. PAD — 꿈은 재생이 아니라생성이다

두 번째 계산 모델은 Deperrois·Petrovici·Senn·Jordan 의 eLife 논문 (2022), 줄여서 **PAD** 다. 출발점은 관찰 하나다 — 대부분의 REM 수면 꿈은 겪은 적 없는 장면을 보여 준다. 꿈은 재생이 아니다. PAD 는그 사실을 잡음이 아니라 **계산의 원재료**로 삼는다.

한 사이클은 세 위상이고, 논문의 표현으로 세 상태는 **오직목적함수와 외부 입력의 유무에서만 다르다**.

- **Wake** — 실제 입력을 encoder 에 통과시켜 얻은 latent(입력을압축한 벡터) 을 두 칸 짜리 임시 buffer 에 쓰고, 동시에 재구성 품질로파라미터를 갱신한다.
- **NREM** — buffer 의 latent 을 그림으로 되돌린 뒤 **가림을씌우고**, 그 훼손된 그림을 다시 encode 한 결과가 원래 latent 과같아지도록 encoder 만 갱신한다.
- **REM** — 현재와 직전 사이클의 latent 두 개와 가우시안 noise 를섞어 새 벡터를 만들고, 그것으로 그린 그림을 판별기에게 “내부에서 만든것” 으로 판정하게 하면서 **생성기 쪽에는 부호를 뒤집어** 갱신한다.

기호 풀이 encoder = 입력을 latent 벡터로 압축하는 신경망. generator = latent 에서그림을 되돌리는 신경망. discriminator(판별기) = 그림이 실제 입력인지내부 생성물인지 판정하는 신경망. **부호를 뒤집는다** = 같은 gradient 를 반대 방향으로 적용한다는 뜻이며, 표준 GAN 학습이 하는 일 그자체다.

핵심 PAD 가 자기 기여로 내세우는 것은 생물학적 예측이다 — **각성과 REM 에서 동일한 국소 오차가 반대 방향의 가중치 변화를 낳는다**. 계산쪽에서 보면 이것은 새 연산도 새 커널도 아니다. 그래서 이 사실은 약점이나아니라 **이송 가능성의 지표**로 읽어야 한다. 생물학에서 “각성과수면은 다른 종류의 가소성을 쓴다” 로 서술되는 것이 계산 쪽에서는 “오프라인 라운드는 목적함수와 부호만 다르다” 로 환원된다. 이 라인이생물학에서 가져올 수 있는 것은 대개 **스케줄과 목적함수의배치**이지 새로운 갱신 primitive 가 아니다.

평가는 표현의 **선형 분리도**다 — 모델을 열린 뒤 latent 위에 선형분류기 하나만 학습시켜 얻은 테스트 정확도.

표 E03-4. PAD 와 병리 조건의 최종 선형 분리도 (%) [PAD Appendix 1—table 1].

데이터셋	PAD	기억 혼합			
		없음	REM 없음	NREM 없음	Wake 만
CIFAR-10	58.25 ± 0.70	53.87 ± 0.85	46.00 ± 0.43	58.00 ± 0.34	42.25 ± 0.54
SVHN	78.92 ± 0.40	60.87 ± 5.07	42.30 ± 1.51	73.25 ± 0.22	41.93 ± 0.65

REM 을 빼면 두 데이터셋 모두 크게 무너진다. 논문의 결론은 여기서 나온다 — 표현을 만드는 것은 오프라인이라는 **사실 자체**가 아니라오프라인에서 **무엇을 생성하는가**다.

이제 이 권의 초점이다. **저자 자신의 통제가 논문 제목의 성분을지지하지 않는다**.

한계 세 가지를 함께 읽어야 한다. 첫째, PAD의 REM은 여러 기억을 섞은 것과 자발 활동(noise)을 함께 주입해 구동되는데, 저자는 통제 실험에서 **혼합이 필요 없다**고 보고한다 — “기억의 조합에 자발 활동을 더한 것과 자발 활동만 쓴 것 사이에 유의한 차이를 관측하지 않았다” [PAD Discussion]. 표의 “기억 혼합 없음” 열은 혼합만 없앤 조건이 아니라 **noise까지 없애고 단일 기억만 되돌린 조건**이다. 두 결과를 합치면 REM에서 결정적인 성분은 기억의 혼합이 아니라 **자발 활동의 주입**이다. 논문 제목이 가리키는 성분은 통제 대비 효과를 보이지 않았다. 둘째, **단계 순서도 무관하다** — “sleep 위상의 순서를 바꿔도 성능에 유의한 차이를 보이지 않는다” [PAD Discussion]. 생물학쪽에서는 NREM → REM 순서가 중요하다고 가설되지만 모델은 그것을 재현하지 않는다. 셋째, **절대 대조군은 손봐진 상태로 비교된다** — REM을 뺀 모델은 성능이 떨어진다는 이유로 손실 함수에 잡음 항이 추가되고 NREM 손실이 0.5 배로 축소된 상태에서 학습되었다. 논문은 이 조정을 공정 비교를 위한 것이라 밝히며 숨기지 않지만, 그 열은 “REM을 뺀 PAD”가 아니라 **“REM을 빼고 저자가 손본 다른 모델”**이다.

여기에 두 가지를 더 얹는다. CIFAR-10에서 “NREM 없음”은 58.00 ± 0.34 로 PAD의 58.25 ± 0.70 과 오차 안에서 같다. NREM의 기여는 깨끗한 이미지의 분리도가 아니라 가림이 있을 때의 견고성에서만 나타난다. 그리고 표현품질의 지표는 **선형 분류기 정확도 하나뿐**이며 downstream 과제는 측정되지 않았고, 그 하나뿐인 지표의 절대값도 CIFAR-10에서 약 59%로 낮다. PLOS와 마찬가지로 PAD에도 **검정 이름도 p-값도 없다** — 이 라인의 근거로 인용되는 바로 그 두 논문이, 유의성 주장을 검정 없이 한다.

핵심 PAD는 **기제의 존재 증명**이지 성능 주장이 아니다. 절대 성능을 근거로 이 라인의 유망함을 말하는 데 이 논문을 쓰는 것은 오용이다. 그리고 sleep 단계를 개수와 순서까지 묘사하려는 LLM 설계에 대해 **이 논문은 순서의 필요성을 지지하지 않는다**. §9에서 오프라인 위상의 개수를 판별기로 쓴 것은 은유가 이송되었는지를 재는 지표이지, 개수와 순서가 성능을 낳는다는 주장이 아니다.

비용은 여기서도 비어 있다. PAD는 FLOPs·GPU 시간·메모리를 어느 것도 보고하지 않는다. 상태 용량은 구조적으로 **2칸**으로 고정되고 보존 기간은 정확히 한 사이클이다 — 매 사이클 덮어쓰므로 누적되는 것이 없다. ρ 에 가장 가까운 관측은 방향성뿐이다: 오프라인 위상 하나(NREM)를 빼면 여러 epoch(데이터셋을 한번씩 다 도는 단위) 뒤에 선형 분리가 **떨어지기 시작한다**.

자세한 것은 ch08 §08.4.

12. 은유는 어디서 깨지는가

이 권의 결론이다. 두 편의 계산 모델에서 실제로 이송되는 것과, 이름만 옮겨 가고 근거는 남는 것을 가른다.

먼저 이송되는 쪽이다. 다음 네 문장은 각각 적어도 하나의 계산 모델에서 실증되었다. 표 E03-5. 생물학 두 편에서 이송 가능한 구조적 주장.

이송되는 주장	근거
오프라인 라운드의 학습 집합은 데이터셋일 필요가 없다 . 모델을 noise 로 돌려 만든 스트림으로 충분했고, 과거 경험은 가중치 안에만 있어도된다	PLOS Sleep Results
각성 → 오프라인 경계를 넘는 보조 상태는 스칼라 하나로 줄어들 수있다	PLOS Sleep 통제 실험
오프라인의 가치는 “오프라인” 이라는 스케줄이 아니라 거기서 무엇을 생성하는가 에서 나온다. 단 결정적인 성분은 기억의 혼합이 아니라 자발 활동의 주입이다	PAD 절제표 + 저자 자신의 통제
오프라인 위상이 하나가 아니라 목적함수가 다른 둘일 때 서로 다른성질 (정확도 / 섭동 견고성) 이 갈라져 나온다	PAD 절제표

이제 이송되지 않는 쪽이다. 이쪽이 더 길다.

PLOS 모델에는 loss 도 gradient 도 optimizer 도 backward pass 도없다. 갱신은 국소 Hebbian 곱이며 learning rate 는 독립된 존재조차아니다. 그 기제는 transformer 가 갖지 않은 장치들에 기댄다 — spike timing 이라는 상관 신호, 뉴런마다의 목표 발화율 항상성, 시냅스 열의 합을보존하는 정규화, 균형 잡힌 전방 억제. 논문이 sleep 의 우위를 설명할 때기대는 것이 바로 그 정규화 제약이고, 제약을 빼면 그 설명에는 대체물이없다. 규모도 이송되지 않는다 — 계속 학습되는 시냅스 6272 개, 과제 2 개, 보호 밴드 0.20 이다.

PAD 는 gradient 를 쓴다. 그러나 **언어도 토큰도 attention 도 없고**, 저장은 2 칸이며, 평가 지표는 선형 분류기 하나다.

그리고 두 논문에 공통된 침묵이 가장 무겁다.

한계 두 논문 모두 **replay 와 dream 의 내용이 의미상 옳은지를 한 번도검사하지 않는다**. 이미지 모델에서는 그 침묵이 치명적이지 않다 — 그림이조금 이상해도 표현은 는다. LLM 에서 이 검사 누락에 대응하는 실패는 다른종류다. **유창하지만 거짓인 생성물로 배포 가중치를 갱신하는것이며**, 이것이 Θ -경로의 중심 위험이다 (→ ch06). 생물학 쪽 두 편은 그위험에 대해 아무 말도 하지 않는다.

이송의 반대편, 즉 이름만 옮겨 간 쪽의 표본이 있다. 이 라인에서 가장 자주인용되는 Sleep-time Compute 논문은 sleep 이라는 단어를 **170 회** 쓰면서 생물학 문헌을 **한 편도 인 용하지 않는다**. 그단어를 도입하는 문장은 순전히 운영적이다 — 모델이 유휴 상태일 때 미리 추론해 둔다는 뜻이다. 반대쪽 끝에는 Language Models Need Sleep 이 있다. 한 절 전체를 SWS/REM 분업과 hippocampus – neocortex 대화에 쓰고 자기기제를 그 위에 얹는다.

시스템 모델링 관점 두 용법의 차이는 서빙 운영으로 옮기면 즉시 드러난다. **유휴의뜻으**로 쓰는 sleep 은 warm-up 이나 사전 컴파일과 같은 자리에 있다 — 한가할 때 미리 해 두 고, 배포 아티팩트는 그대로다. 틀리면 **캐시를무효화하면 끝난다**. 반면 **consolidation 의 뜻으로** 쓰는 sleep 은모델 재배포 1 회와 같은 자리에 있다 — 끝나고 나면 서빙되 는가중치가다른 것이 된다. 틀리면 **롤백해야 하고**, per-user delta 가 생기면 shared-weight batching 이 깨지며, 라운드마다 회귀 스위트를 돌려 ρ 를 재야 한다. 상각 구조도, 배치 가능성도, 롤백 절차도 같지 않다. **같은 단어 하나가 이 구분을 덮고 있다**. 어떤 논문이 “sleep” 이라고 쓸 때 참고문헌에 신경과학이 있는지부터 확인하는 것이가장

값싼 판별이다.

자세한 것은 ch08 § 08.5.

요약

- 망각은 Θ -경로만의 문제다. E 는 덮어쓰면 되고 W 는 사라지지만, 배포 가중치는 라운드마다 다른 함수가 되어 남는다.
- 망각은 버그가 아니라 갱신식이 정의하는 동작이다. 학습 집합에 과거가 없으면 갱신은 과거를 지킬 압력을 한 번도 받지 않는다.
- 대응은 셋 중 하나를 고치는 일이다 — 학습집합 (rehearsal·generative replay), 목적함수 (EWC), 갱신방향 (GEM). 네 방법 모두 **서빙 비용은 똑같고**, 차이는 전부 오프라인 라운드 안에 있다.
- EWC의 기억 비용은 step 당 $2|\Theta|$ 원소의 대역폭 세금이며 **라운드 수와 무관하다**. 매일 자는 시스템에서는 이 상수성이 요구사항이다. 다만 **EWC 논문에는 결과표가 없고** 이후 문헌의 모든 EWC 수치는 재구현에서 나온 값이다.
- GEM은 실제 예제를 저장하고 step 비용이 라운드 수에 선형이다. EWC의 용량 논거를 논박한 것이 아니라 다른 자원으로 옮긴 것이다. GEM의 진짜 기여는 알고리즘이 아니라 성능 행렬이라는 **계기**다.
- generative replay는 저장을 계산으로 바꾸지만 재귀가 이중이다. 그 표류를 켜 놓는 과제가 하나뿐인 예비 실험 하나이며, 자기 상한대조군을 이긴 적이 없고 비길 뿐이다.
- ρ 는 라운드 하나가 이미 배운 과제들에 입힌 상대 손실이다. 같은 실행에서도 정의를 바꾸면 값이 4배 넘게 벌어지므로, **정의를 밝히지 않은 ρ 는 숫자가 아니다**.
- CLS는 속도 논증이 아니라 간섭 논증이고, 그 처방인 interleaving은 과거 데이터의 영구 저장을 요구한다. 그 간극이 Θ -경로 전체의 문제의식이다.
- PLOS 실험은 과거 데이터를 한 번도 다시 쓰지 않고 옛 과제를 0.70에서 0.70으로 지켰다. 그러나 데이터를 저장하는 고전 interleaving이 0.68/0.65로 표준편차 안에서 같고, 논문은 우위를 실증이 아니라 **이론적 근거**로 돌린다. **실증된 것은 정확도 우위가 아니라 저장의 제거**다.
- PAD는 오프라인의 가치가 생성물에 있음을 보이지만, 저자 자신의 통제에서 **기억의 혼합은 필요하지 않고** (자발 활동만으로 차이 없음) **단계의 순서도 무관하다**. 절제 대조군은 손봐진 상태로 비교되고, 두 생물학 논문 모두 검정 없이 “유의하게”를 쓰며 계산 비용을 어느 형태로도 보고하지 않는다.
- 이송되는 것은 스케줄과 목적함수의 배치이지 새 갱신 primitive가 아니다. 두 논문 모두 replay·dream 내용의 진위를 검사하지 않으며, LLM에서 그 누락에 대응하는 실패가 이 경로의 중심 위험이다.

자가 점검

- 왜 E 와 W 에는 catastrophic forgetting이 없고 Θ 에만 있는지 한 문장으로 말할 수 있다.

- rehearsal · generative replay · EWC · GEM 이 각각 갱신식의 어디를 고치는지 지목할 수 있고, 네방법의 서빙 비용이 왜 같은지 설명할 수 있다.
- EWC 의 비용이 라운드 수와 무관하고 GEM 의 비용이 라운드 수에 선형이라는 사실이, 매일 도는 sleep 라운드 운영에서 무엇을 뜻하는지 말할 수 있다.
- 어떤 논문이 “망각을 줄였다” 고 주장할 때 (1) 어느 과제의 어느 지표를 (2) 어느 라운드에서 (3) 무엇에 대해 정규화해 줬는지 세 질문으로 검증할 수 있다.
- 표 E03-3 에서 PLOS 실험이 실증한 것과 실증하지 않은 것을 각각 한문장으로 말할 수 있다.
- PAD 저자 자신의 통제가 논문 제목의 성분과 단계 순서 둘 다를 지지하지않는다는 사실을 지적할 수 있다.
- (Rosetta) 어떤 논문의 “sleep” 이 **warm-up / 사전컴파일** 자리에 있는지 **모델 재배포 1 회** 자리에 있는지를참고문헌으로 판별할 수 있다. 앞의 것은 캐시 무효화 문제이고 뒤의 것은배포 문제다.

다음 권으로

이 권은 두 가지를 열어 둔 채 끝난다. 첫째, 오프라인 라운드를 **몇번이나** 반복할 수 있는가 — PLOS 실험은 과제 두 개에서 멈췄고 ρ 를재지 않았으며, PAD 는 오프라인 위상 하나를 빼면 반복 라운드가 성능을 깎기시작한다는 방향성만 보고한다. 둘째, 더 앞선 질문으로 **가중치에애초에 얼마나 넣을 수 있는가**. 파라미터당 몇 bit 가 저장되는지에 상한이있다면, “저장을 계산으로 바꾼다” 는 이 권의 교환에도 상한이 있다. 다음권이 그 용량 질문을 받는다.